



Tarea 1

Entrega

- **Fecha y hora:** Viernes 5 de septiembre del 2025, a las 22:00
- **Lugar:** Repositorio individual en la organización del curso en GitHub, main branch.
- **Link de Invitación**

Objetivos

- **Comprender** el funcionamiento de una API a partir de documentación.
- **Crear** un programa que pueda manejar requests y consumir una API
- **Producir** documentación efectiva y clara que permita el entendimiento del proceso realizado

Descripción

En esta tarea, explorarán cómo integrar y utilizar una API (Interfaz de Programación de Aplicaciones), una habilidad muy útil en el desarrollo de software actual. Frecuentemente, esto les permitirá aprovechar recursos preexistentes en lugar de desarrollar soluciones desde cero, optimizando así el uso de su tiempo y recursos. Esta práctica no solo les ahorra esfuerzo, sino que también les introduce a la amplia gama de herramientas y servicios disponibles en la nube, preparándolos para enfrentar desafíos similares en entornos profesionales.

Para este propósito, deberán hacer uso de la [PokeAPI](#), **diseñada por el equipo docente** para simular la información de *marketplace* para venta y compra de Pokemons. Esta API les suministra datos detallados sobre la aplicación, permitiéndoles crear un prototipo funcional de una aplicación. Tendrán la oportunidad de crear un usuario y empezar a comprar y vender pokemons, con una gran variedad de tipos, niveles y precios. Junto a esto, deberán hacer un pequeño manejo de inicio de sesión para los usuarios. La información obtenida de la API deberá ser presentada a los usuarios a través de una interfaz desarrollada en React, asegurando una experiencia interactiva y atractiva.

Especificaciones

Para esta tarea se espera que puedan recrear un marketplace utilizando componentes de React, a partir del código base entregado. Para lograr esto, deberán utilizar la [documentación](#) de la PokeApi creada por el equipo docente, en donde podrán encontrar los endpoints que deben consumir para esta tarea.

Junto a ello, la URL que deberán usar para conectarse con la API y consumirla es la siguiente:

- <https://t1-back-2025-2-s1.onrender.com/>

Pages

Se les entregarán 4 archivos en la carpeta de pages, estos serán utilizados para mostrar las vistas de la página.

- **Landing.jsx:** es la landing page de su aplicación. Aquí deberán colocar un mensaje de bienvenida a su página y mencionar qué cosas puede hacer un usuario. ¡Hagan volar su creatividad!

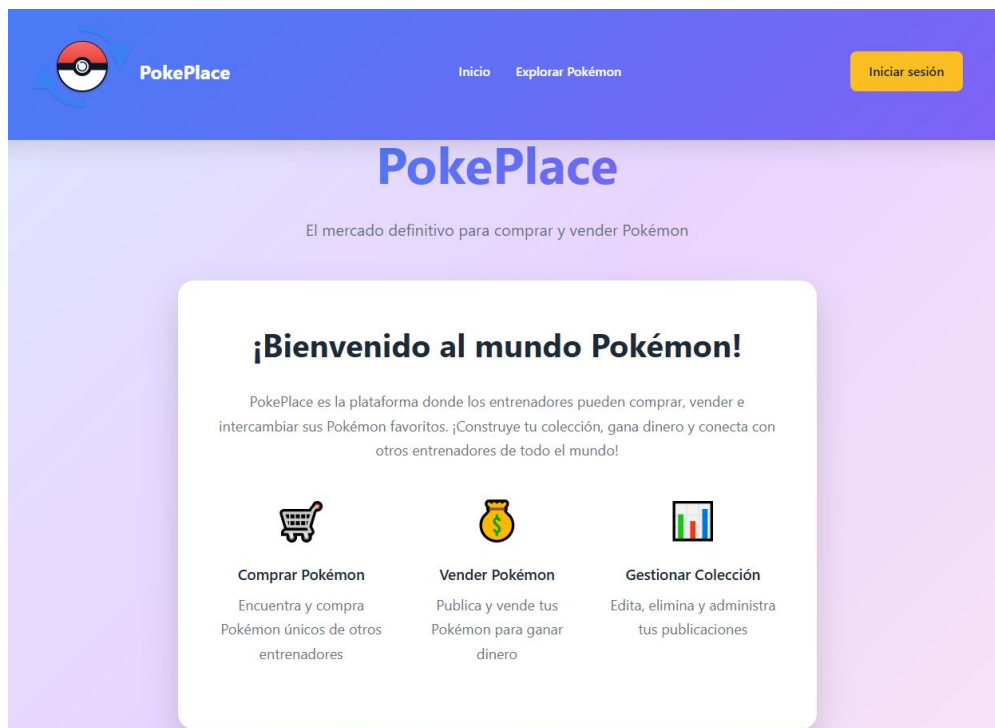


Figura 1: Ejemplo representativo de la página *Landing*.

- **Login.jsx:** Esta página sirve para que un usuario pueda iniciar sesión en la aplicación web y de esta forma pueda vender o comprar pokemons. Para efectos prácticos, en esta tarea, si el usuario no existe al logear, este se crea automáticamente (esta lógica ya está implementada en la API).

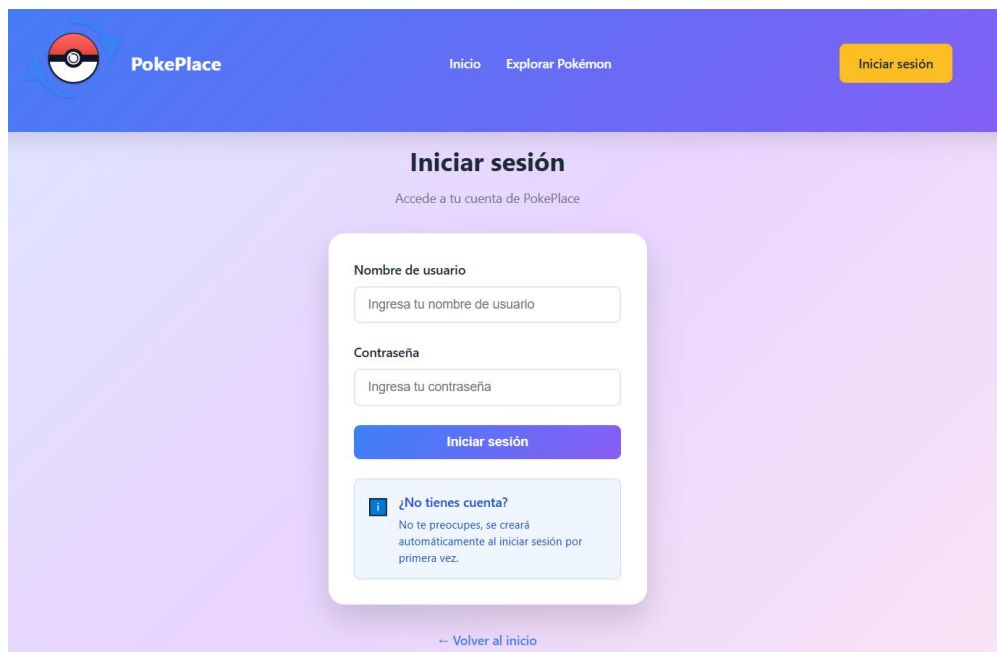


Figura 2: Ejemplo representativo de la página *Login*.

- **MainPage.jsx:** Esta es la página principal de su aplicación. Aquí deberán mostrar todos los pokemons que se encuentran a la venta, utilizando paginación. Los pokemons deben estar dispuestos de a pares en cada fila. Los pokemons se muestran usando el componente PokemonCard y al 'Examinar' se debe abrir el componente PokemonCardModal detallados más adelante.

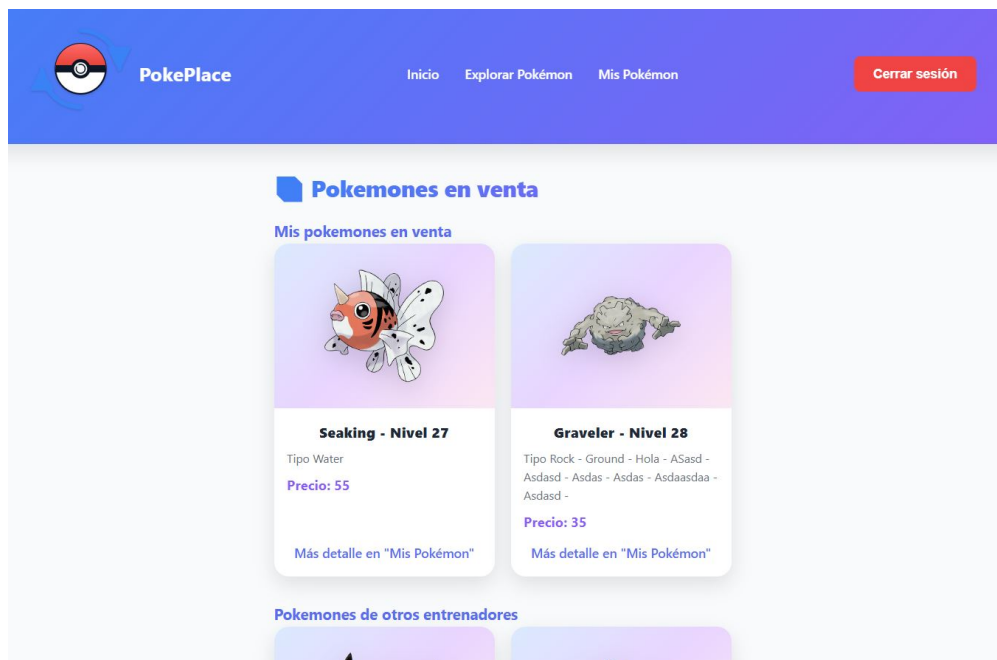


Figura 3: Ejemplo representativo de la página *MainPage* (1 de 2). Se muestran 'Mis pokemons en venta'

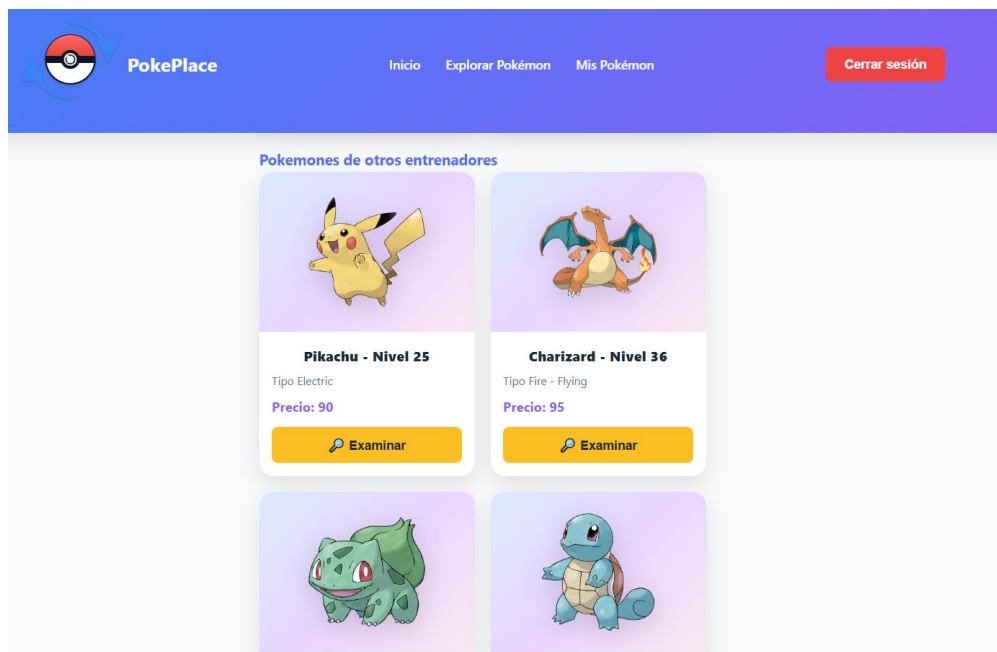


Figura 4: Ejemplo representativo de la página *MainPage* (2 de 2). Se muestran los Pokemons en venta por parte de otros entrenadores

- **MyPokemonsPage.jsx:** En esta página se deben mostrar todos los pokemons que un usuario ha adquirido o creado, utilizando el componente `PokemonCard` que se detallará más adelante. Al presionar 'Examinar', se abrirá el componente `PokemonCardModal` detallado más adelante. Además, esta vista debe poseer:
 - Nombre de usuario
 - Saldo del usuario
 - Botón para crear pokemon

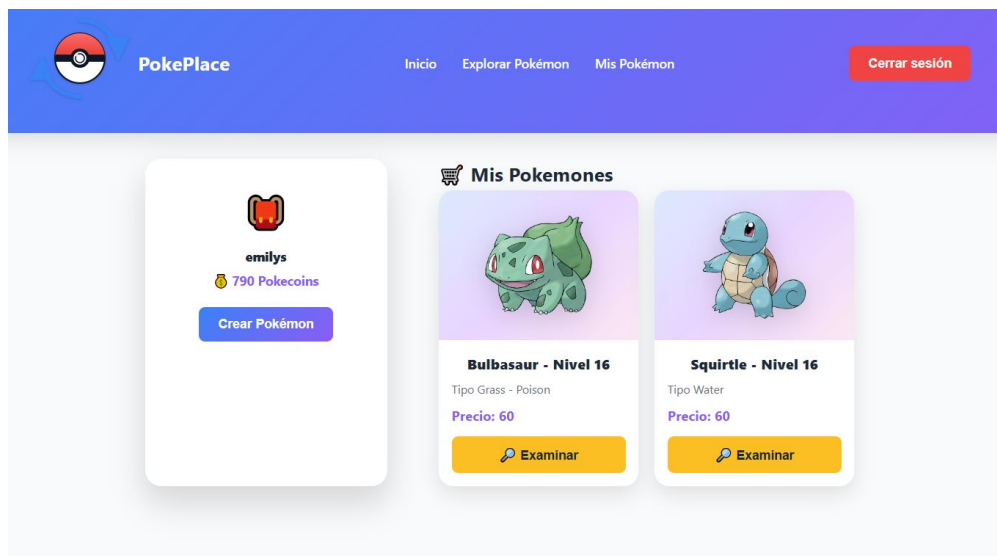


Figura 5: Ejemplo representativo de la página *Mis Pokemons*

Components

Su página debe contar mínimo con estos componentes.

- **Navbar:** Componente que permite la navegación entre las distintas vistas de la aplicación. Deberán considerar que esta Navbar se modifica según un usuario haya iniciado sesión o no.
- **CreatePokemonModal:** Componente utilizado para crear un pokemon. Este componente, al ser un formulario, debe incluir validación de datos.
 - Nivel: entero entre 1 y 100
 - Precio: entero entre 0 y 2000
 - Imagen: url

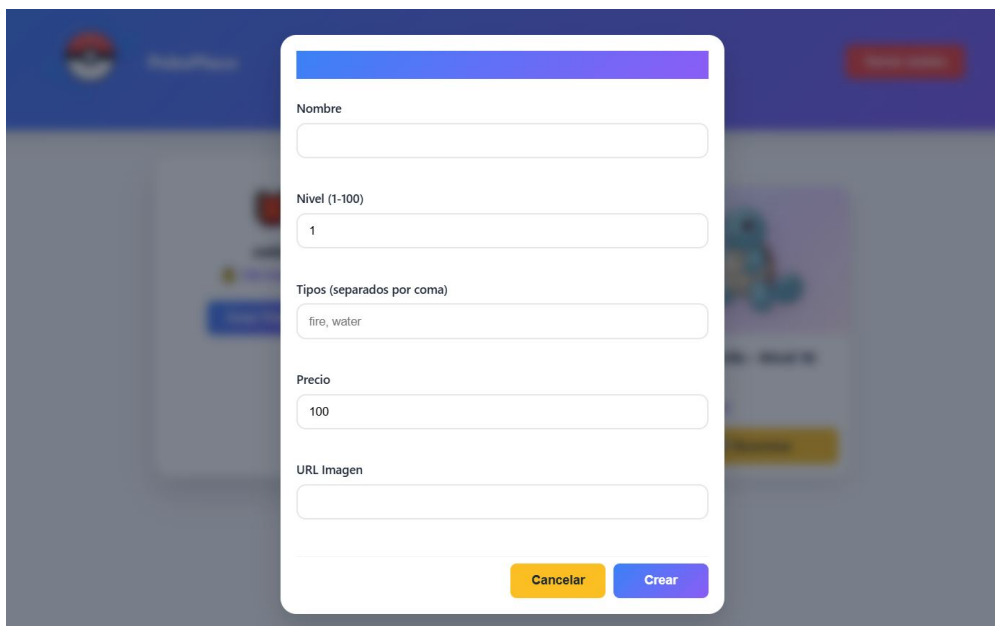
A screenshot of a web application showing a modal form titled 'CreatePokemonModal'. The form is white with a blue header bar. It contains five input fields: 'Nombre' (empty), 'Nivel (1-100)' (containing '1'), 'Tipos (separados por coma)' (containing 'fire, water'), 'Precio' (containing '100'), and 'URL Imagen' (empty). At the bottom right of the form are two buttons: 'Cancelar' (yellow) and 'Crear' (blue). The background is a blurred image of a Pokemon game interface.

Figura 6: Ejemplo representativo del componente *CreatePokemonModal*.

- **EditPokemonModal:** Componente utilizada para editar la información sobre un pokemon. Esta al ser un formulario, debe incluir validación de datos.
- **PokemonCard:** Este componente se encarga de mostrar la información resumida de un pokemon. El botón de examinar abrirá el componente *PokemonCardModal* detallado más adelante.



Figura 7: Ejemplo representativo del componente *PokemonCard*.

- **PokemonCardModal:** Este componente se encarga de mostrar la toda información de un pokemon. A diferencia de *PokemonCard*, este componente debe indicar quién es el dueño del pokemon y mostrar todos los tipos del pokemon. Además, este componente se usa tanto en la main page como en *MyPokemonsPage*.

En la main page debe permitir comprar un pokemon, tal como se ve en la siguiente figura:

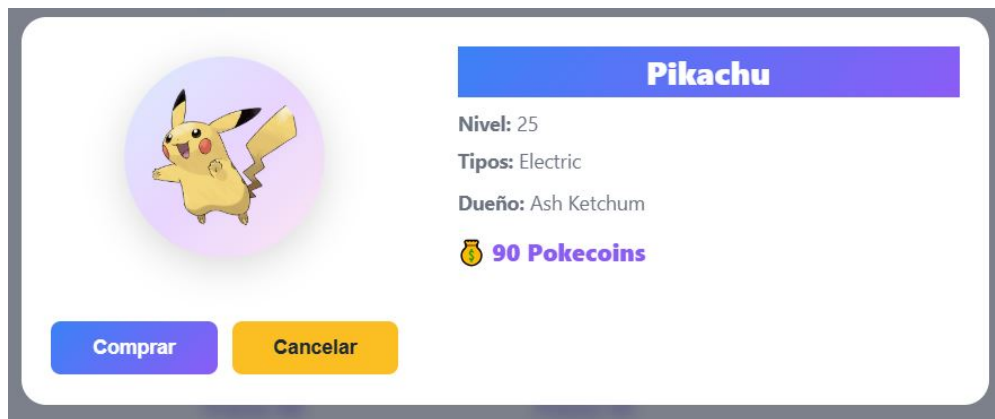


Figura 8: Ejemplo representativo del componente *PokemonCardModal* en mainpage.

En MyPokemonsPage debe permitir las siguientes acciones:

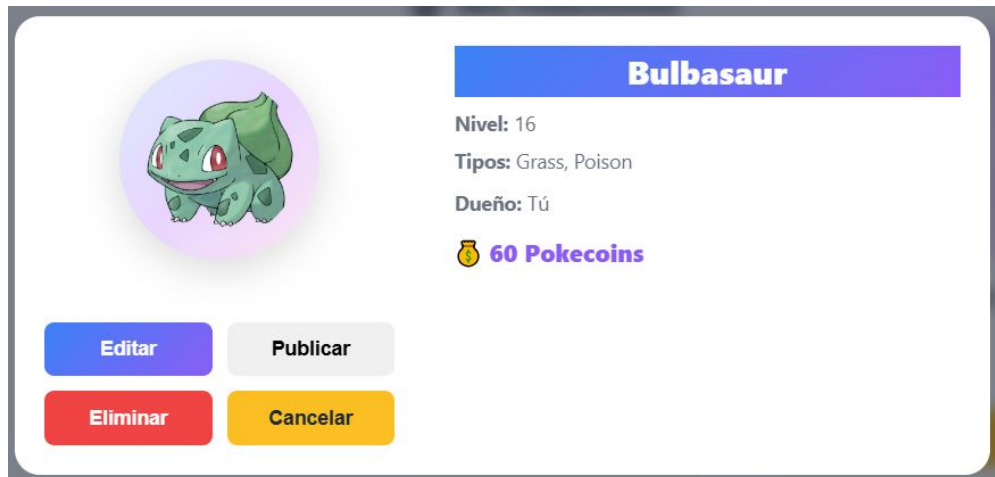


Figura 9: Ejemplo representativo del componente *PokemonCardModal* en Mis Pokemons.

Importante: se permite separar el componente *PokemonCardModal* en dos para mayor simplicidad.

- **ConfirmActionModal:** componente que se encarga de mostrar un mensaje de confirmación para las acciones posibles con los pokemons. Todas las acciones deben ser cubiertas por el modal de confirmación. Es decir, al crear, comprar, publicar, eliminar o editar un pokemon o cancelar una publicación, se debe solicitar la confirmación.

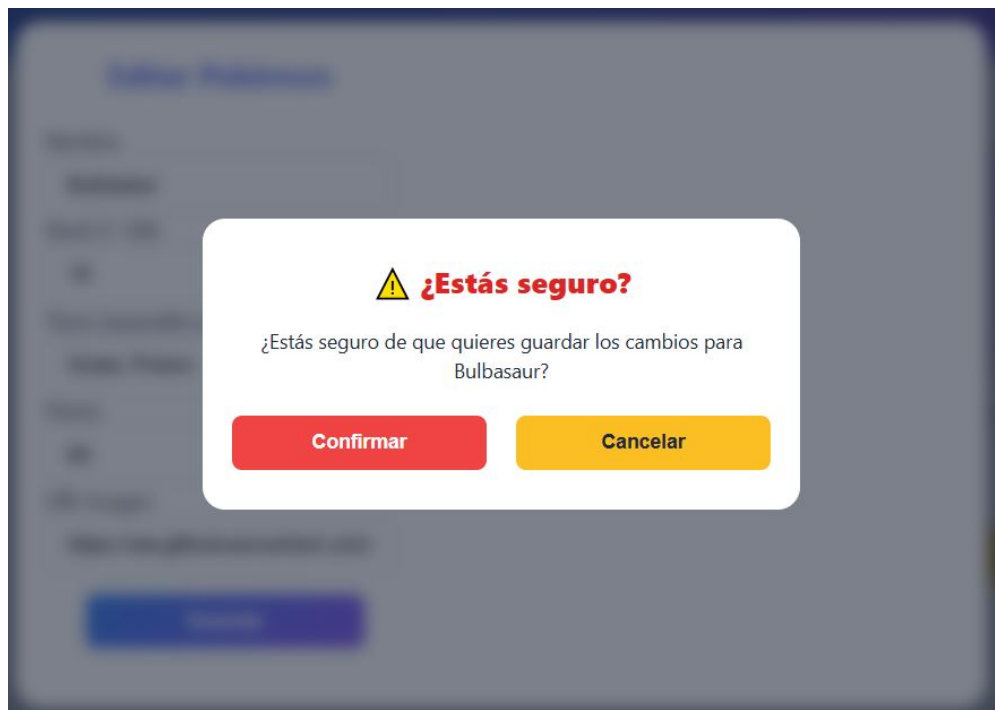


Figura 10: Ejemplo representativo del componente *ConfirmActionModal* al editar un pokemon.

- **SuccessMessageModal:** componente que se encarga de mostrar un mensaje de que la acción se ha realizado con éxito. Todas las acciones posibles deben tener un mensaje de éxito asociado.

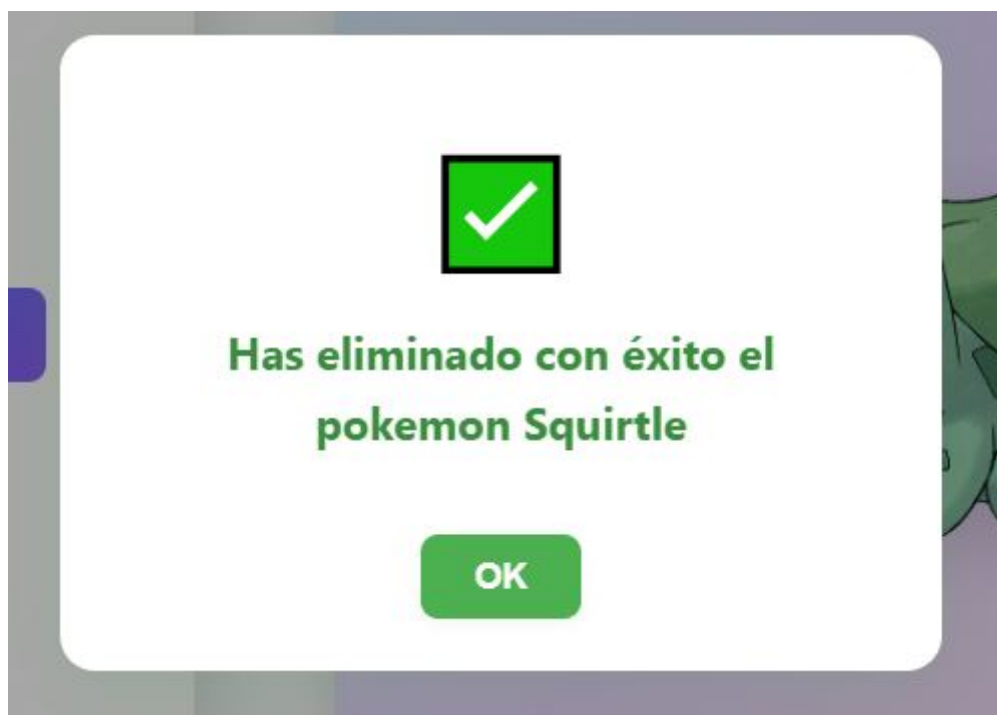


Figura 11: Ejemplo representativo del componente *SuccessMessageModal* al eliminar un pokemon

- **ErrorMessageModal:** componente que se encarga de mostrar un mensaje de error ante posibles errores en las acciones.

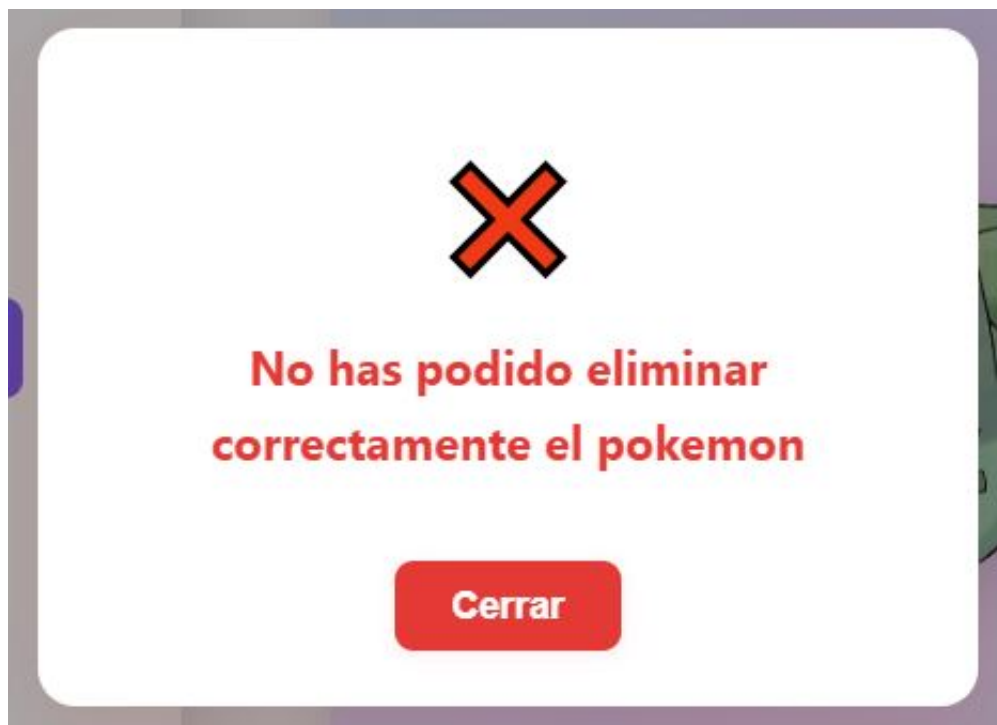


Figura 12: Ejemplo representativo del componente *ErrorMessageModal* al no poder eliminar un pokemon.

Consideraciones Importantes

A continuación, se presentan algunas directrices **importantes** para el desarrollo de la tarea:

- **Restricciones de Librerías:** Está **estrictamente prohibido** el uso de cualquier librería externa o componentes preconstruídos (bootstrap, material UI, Bulma o parecidos). El incumplimiento de esta regla resultará en una calificación automática de 1.0, sin posibilidad de re-corrección.
- **Pulls en GitHub:** No deben subir el directorio `node_modules`. Además, solo se revisará en la **rama main**. El incumplimiento de esta regla resultará en la reducción en un 50% de su nota final, sin posibilidad de re-corrección.
- **Herramienta de petición HTTP:** se permitirá el uso de `axios` como librería externa para la realización de solicitudes HTTP a la API. También está permitido el uso de `fetch`. Ninguna otra herramienta está permitida. El incumplimiento de esta norma resultará en la reducción del 50% de su nota final, sin posibilidad de re-corrección.
- **Diseño y Código Base:** El código base proporcionado tiene un carácter orientativo y no es obligatorio seguirlo en su totalidad. Sin embargo, todos los `components` y `pages` deben estar correctamente implementados y funcionales. Se permite plena libertad en cuanto al diseño visual, siempre que se cumplan los requisitos funcionales de la tarea.
- **Reinicio Base de Datos:** La base de datos se reinicia cada dos horas, desde las 8:00am a 10pm. En cada reinicio, es posible tener una intermitencia breve del servicio, pudiendo tener demoras cortas o errores 502. Se recomienda esperar un par de minutos o verificar que la documentación se encuentre en línea.
- **Uso de postman:** Se recomienda utilizar postman o Swagger para probar endpoints.
- **Buena práctica de diseño:** todos los modals deben tener un botón para salir de este mismo, ya sea con el botón de cancelar o una cruz en la esquina superior derecha.
- **Video:** a modo de ejemplo, se ha realizado un video explicando cómo debe ser el flujo de su tarea y las funcionalidades que esta debe tener. Para ello, pueden guiarse por el siguiente [video](#). Cabe destacar que el objetivo no es que sigan el mismo diseño. Más bien, el video es simplemente ilustrativo y con fines de explicar de mejor forma los alcances de su tarea.

Interfaz gráfica

Para esta parte, tendrán que mostrar visualmente los resultados de las *requests* descritas en la documentación de la API. Para ello, les proveeremos un código base en React que podrán encontrar en sus respectivos repositorios.

Documentación del proceso

Este ítem tiene como propósito:

- Explicar qué logra hacer su programa y también aquello que no lograron implementar.
- Indicar apropiadamente el proceso para levantar tu aplicación, incluyendo que comandos se deben correr.

Además, en sus respectivos repositorios encontrarán una serie de preguntas teóricas acerca de su tarea y del consumo de APIs en general. Deberán responder estas preguntas de forma más completa y breve posible.

Para un mejor entendimiento sobre cómo escribir la documentación, se les hará entrega de una plantilla (en el README) con la estructura que debe seguir, su descripción y las preguntas que deben responder.

Entregables

Cada entrega deberá incluir los siguientes archivos:

- `README.md` con la documentación de tu programa. Debes mencionar brevemente aquello que pudiste implementar y también lo que no. Debes indicar cómo levantar tu aplicación y responder las preguntas indicadas en el *template* que te facilitamos. Para finalizar, debes explicar claramente cómo funciona cada implementación de tu programa.
- Aplicación en React desarrollada en base al material entregado. Se deben considerar todos los directorios necesarios para poder levantar y correr el programa.

Rúbrica

A continuación, se describe el criterio de cada uno de los ítems de la rúbrica con la que se evaluará esta entrega. La nota se calcula al 50 % considerando un total de **39** puntos.

- **Navbar [2 puntos]:** Se debe incluir la navbar en todas las vistas. Esta debe contener el logo y nombre del marketplace de Pokemon que están creando. Además, debe cambiar según haya o no un usuario logueado, mostrando un botón de Iniciar Sesión o uno de Cerrar Sesión según corresponda. Finalmente, debe permitir ir a la vista de 'Mis pokemons' al estar logueado.
- **Landing Page: [1 punto]:** Página de bienvenida a su marketPlace, en donde muestren en logo en grande y un mensaje creativo de su aplicación.
- **Página Login: [3 puntos]:** Vistas con un formulario que permita ingresar a la aplicación través del nombre de usuario y contraseña. En caso de que la contraseña del usuario ingresado sea errada, debe notificarse. El formulario debe cumplir con estar centrado tanto vertical como horizontalmente.
- **MainPage: [6 puntos]:** Página principal de la aplicación, que debe mostrar todos los Pokemons publicados para la venta. Esta debe cumplir con:
 - Usar componente PokemonCard para mostrar la información del pokemon.
 - Componentes PokemonCards en filas de dos, y cada fila alineada horizontalmente
 - Utilizar paginación, mostrando página actual y número total de páginas.
 - Mostrar pokemons a la venta por parte del usuario logueado. Si no hay usuario logueado, se muestran todos los pokemons de todos los usuarios.
 - Bajo lo anterior, se deben mostrar los pokemons que se encuentran a la venta por parte de todos los otros usuarios.
- **MyPokemonsPage: [2.5 puntos]:** Página de 'Mis pokemons'. Permite mostrar todos los Pokemons publicados para la venta por el usuario logueado. Esta debe cumplir con tener:
 - Nombre de usuario
 - Saldo del usuario, que que varía tras vender/comprar pokemons.
 - Botón para crear más pokemons
 - Mostrar pokemons del usuario logueado usando componente PokemonCard o similar.
 - Utilizar paginación, mostrando página actual y número total de páginas.
- **CreatePokemonModal [3 puntos]:** Este componente es el formulario para crear un pokemon, y debe cumplir con:
 - Tener campos para nombre, nivel, tipos, precio y url de imagen.
 - Validar que no se incluyan campos vacíos (excepto los tipos del pokemon). Además, deben cumplir con formato solicitado.
 - Utilizar el endpoint para crear un pokemon, funcionando correctamente.
- **PokemonCard [2 puntos]** Este componente es aquel que se muestra en la pagina principal y en 'mis pokemons' y debe realizar lo siguiente:
 - Mostrar atributos del pokemon (Imagen, Nombre del pokemon, Nivel, tipos y precio)

- No se deben mostrar más de 2 tipos. En caso de que el pokemon tenga más, deberán colocarse tres puntos '...' simbolizando la existencia de más tipos. La información completa debe poder verse en el componente PokemonCardModal.
 - Mostrar los tipos del pokemon usando el siguiente separador ' - '. (Ej: Tipo: fuego - agua - tierra)
 - Tener botón de 'Examinar'/'Ver', que permita abrir el componente 'PokemonCardModal'.
- **PokemonCardModal [8 puntos]:** Este componente muestra toda la información del pokemon de forma expandida, e incluye además el Nombre del dueño del pokemon. Además, debe cumplir con:
- Contener todos los atributos del pokemon mencionados anteriormente incluyendo nombre del dueño.
 - utilizar endpoint para comprar pokemon al presionar botón de comprar, funcionando correctamente.
 - Poseer botón de editar/publicar-cancelar publicación/eliminar/cancelar al usar este componente en la vista de 'mis pokemons' (estas opciones solamente se deben mostrar si el usuario logueado es el dueño), y deben funcionar correctamente.
 - El botón de comprar solo se puede visualizar si el usuario es un posible comprador, es decir, está disponible cuando no es el dueño del pokemon.
 - Se debe redireccionar al login de la aplicación cuando se presiona el botón de 'comprar' con un usuario no logueado.
- **EditPokemonModal: [1.5 puntos]:** Se espera que este componente sea un modal que incluya un formulario para poder editar un pokemon. Este debe
- Validar que no se incluyan campos vacíos (excepto los tipos del pokemon). Además, deben cumplir con formato solicitado.
 - Tener cargados los datos a editar en el formulario.
 - Utilizar endpoint para editar un pokemon, funcionando correctamente.
- **Diseño [4 puntos]:** Se espera que cumplan con los siguientes aspectos:
- Todos los modals deben tener un botón para salir o cancelar.
 - Pestaña del navegador debe tener logo y nombre de la aplicación
 - La aplicación tiene un diseño intuitivo.
 - El diseño resulta atractivo y bien logrado.
- **Escritura de documentación [6 puntos]:** Se espera que se explique de manera clara, ordenada y suficiente cada parte del programa. Se debe notar un claro entendimiento acerca de cómo se consume una API y de cómo se realizó (y cómo se corre) el código entregado. Además, se esperan respuestas claras y completas explicando las preguntas teóricas asociadas a la tarea.

Bonus

Se puede optar por obtener 5 décimas en la nota final de la tarea por realizar los 3 tipos de mensajes informativos. A continuación se detalla cada uno de ellos:

- **ConfirmActionModal:** Este componente se debe mostrar tras presionar el botón de cada una de las 6 acciones posibles (crear pokemon, editar, eliminar, publicar, cancelar publicación, comprar). Una vez presionado, se debe mostrar un mensaje que solicite confirmar la decisión, para posteriormente ejecutar la acción. Se debe indicar la acción que ocurrirá.
- **SuccessMessageModal:** Este componente se debe mostrar tras realizar con éxito alguna de las 6 acciones posibles (crear pokemon, editar, eliminar, publicar, cancelar publicación, comprar). Se debe indicar la acción que ocurrió.
- **ErrorMessageModal:** Este componente se debe mostrar tras un intento fallido en las 6 acciones posibles (crear pokemon, editar, eliminar, publicar, cancelar publicación, comprar). Se debe indicar la acción que se intentó realizar.

Para obtener el bonus, se deberán tener al menos 14 de las acciones cubiertas. No existe asignación de bonus parcial.

Cupones de Atraso

Tendrán 2 cupones de atraso que podrán ser utilizados en sus tareas. Cada uno equivale a 24 horas adicionales para realizar la entrega de la tarea que elijan, sin perjudicar su nota. Se aconseja destinar un cupón para la tarea 1, y otro para la tarea 2. No obstante, es posible distribuirlos de la forma que estimen conveniente.

Para hacer uso del cupón de atraso, deberán contestar un formulario que se entregará por anuncios de Canvas. Este formulario deben contestarlo antes de la fecha de entrega de su tarea.

Dudas

Cualquier duda que se presente acerca del enunciado debes consultarla en las [issues](#) del repositorio del curso. Recuerda que como equipo docente estaremos atentos para poder ayudarte :)

Notas

No se responderán dudas por correo.

La política de atraso podrán encontrarla en el programa del curso